Лабораторная работа № 2

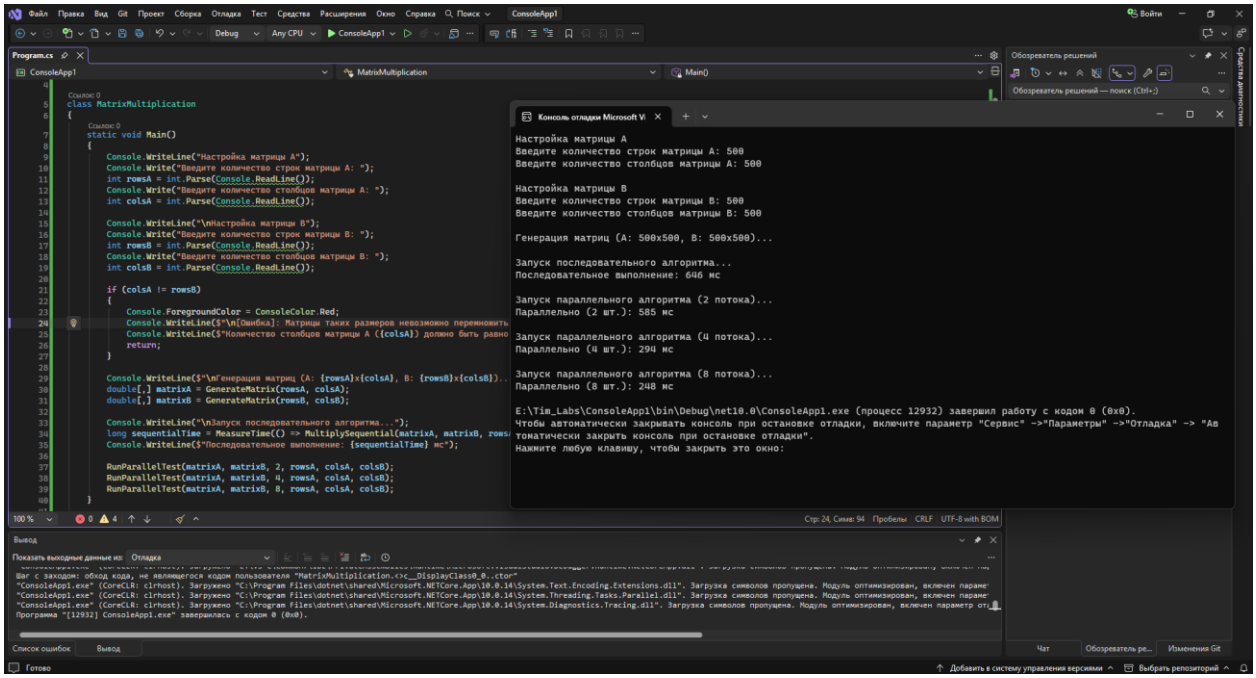
Бадреев Т.А.

гр. 4404

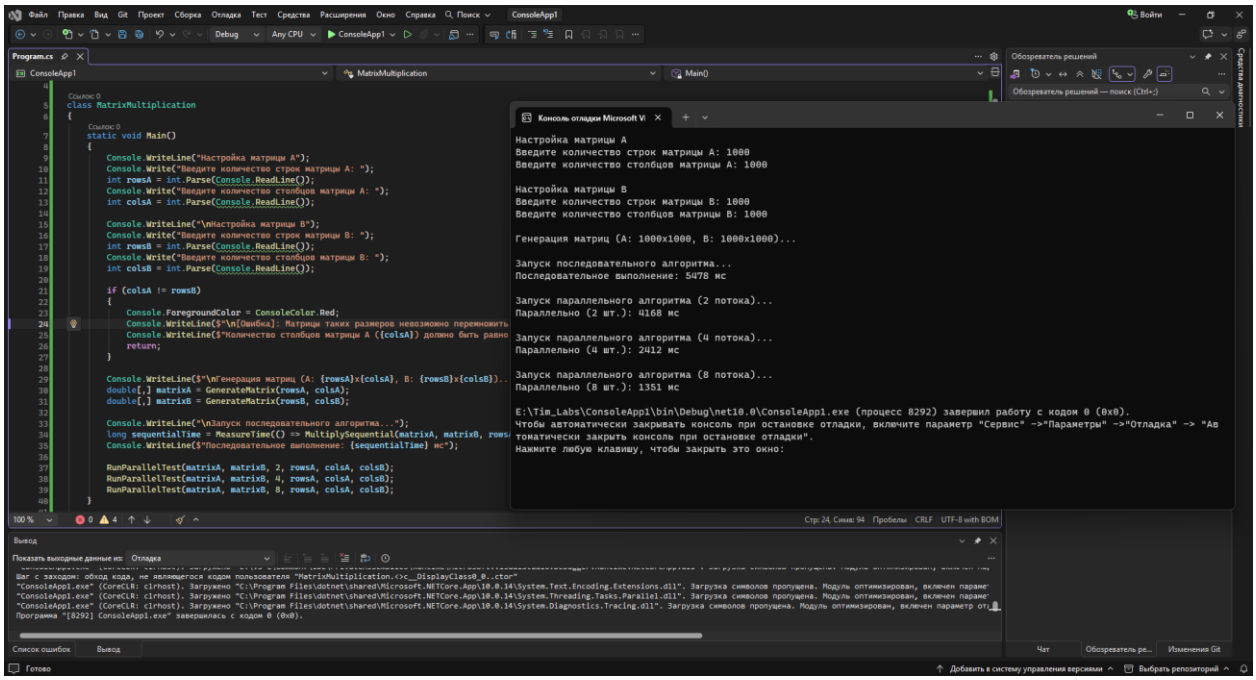
Задача. Напишите программу перемножения больших матриц. Сравните время выполнения последовательной и параллельной программы на 2,4,8 потоках (процессорах).

Ход работы.

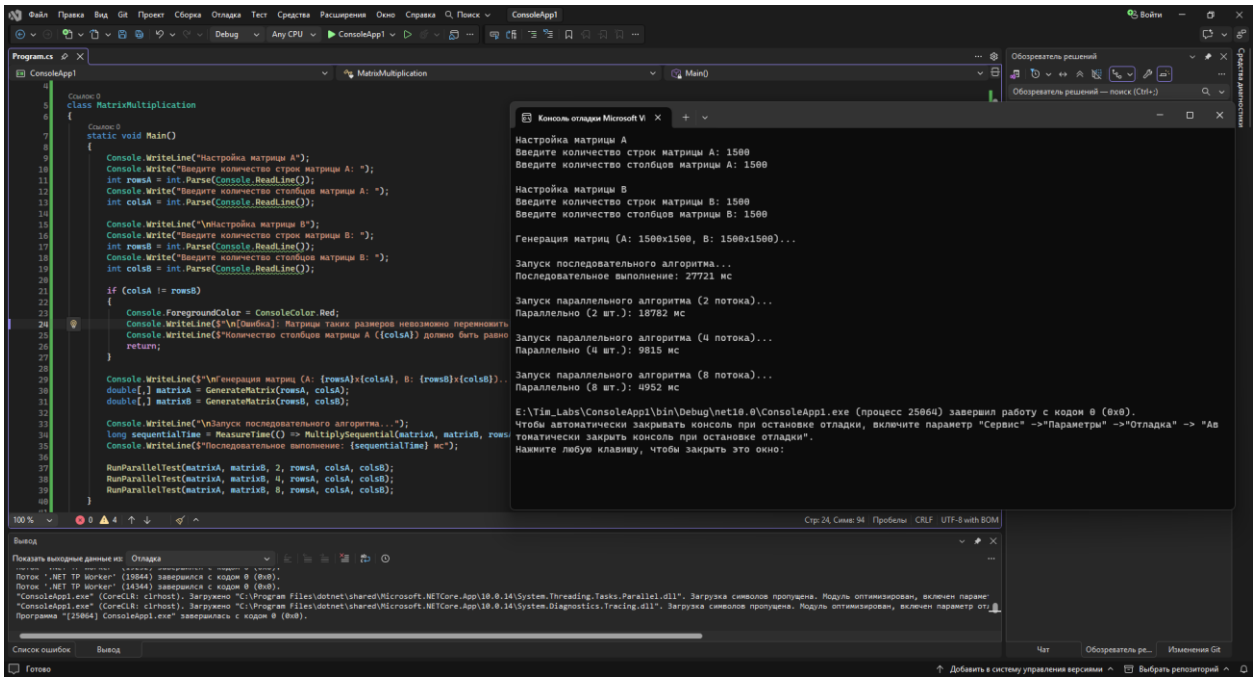
Результат выполнения программы при последовательном выполнении, а также параллельном выполнении при 2,4,8 потоках при размерностях 500x500:



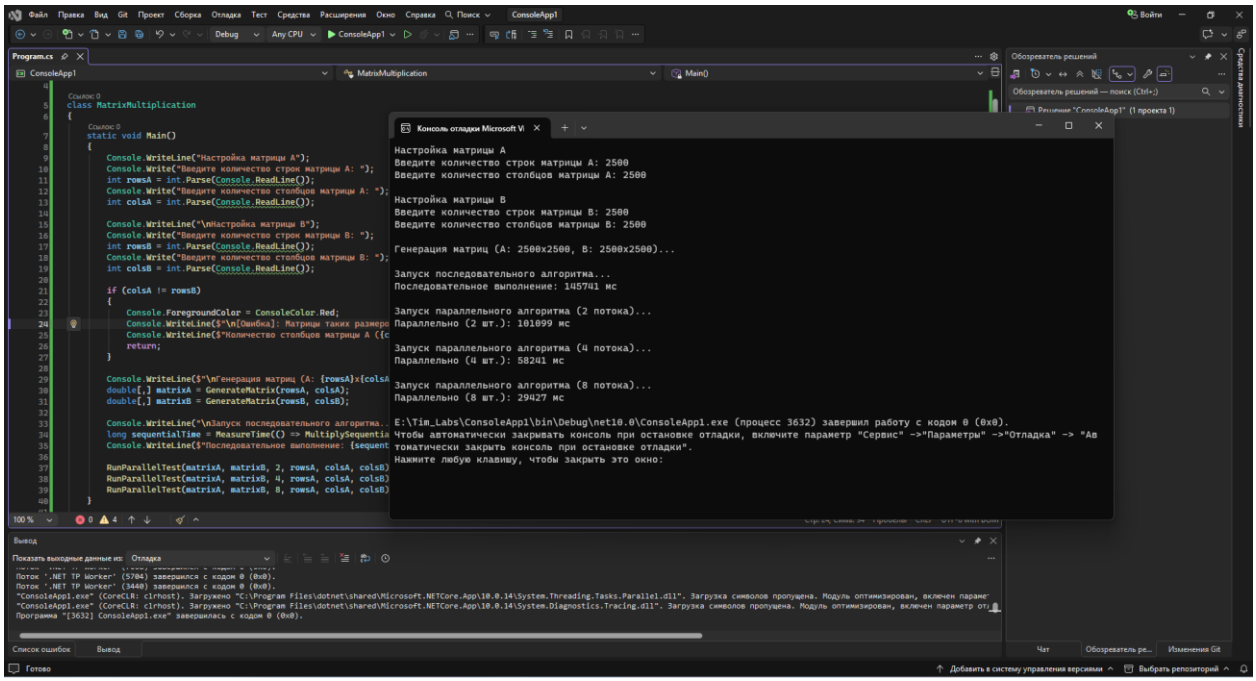
Результат выполнения программы при последовательном выполнении, а также параллельном выполнении при 2,4,8 потоках при размерностях 1000x1000:



Результат выполнения программы при последовательном выполнении, а также параллельном выполнении при 2,4,8 потоках при размерностях 1500x1500:



Результат выполнения программы при последовательном выполнении, а также параллельном выполнении при 2,4,8 потоках при размерностях 2500x2500:



Результаты в виде таблицы:

Размерность	Последовательно (1 поток)	2 потока	4 потока	8 потоков
500 x 500	646 мс	585 мс	294 мс	248 мс
1000 x 1000	5478 мс	4168 мс	2412 мс	1351 мс
1500 x 1500	27721 мс	18782 мс	9815 мс	4952 мс
2500 x 2500	145741 мс	101099 мс	58241 мс	29427 мс

График зависимости времени выполнения от размерности матриц:

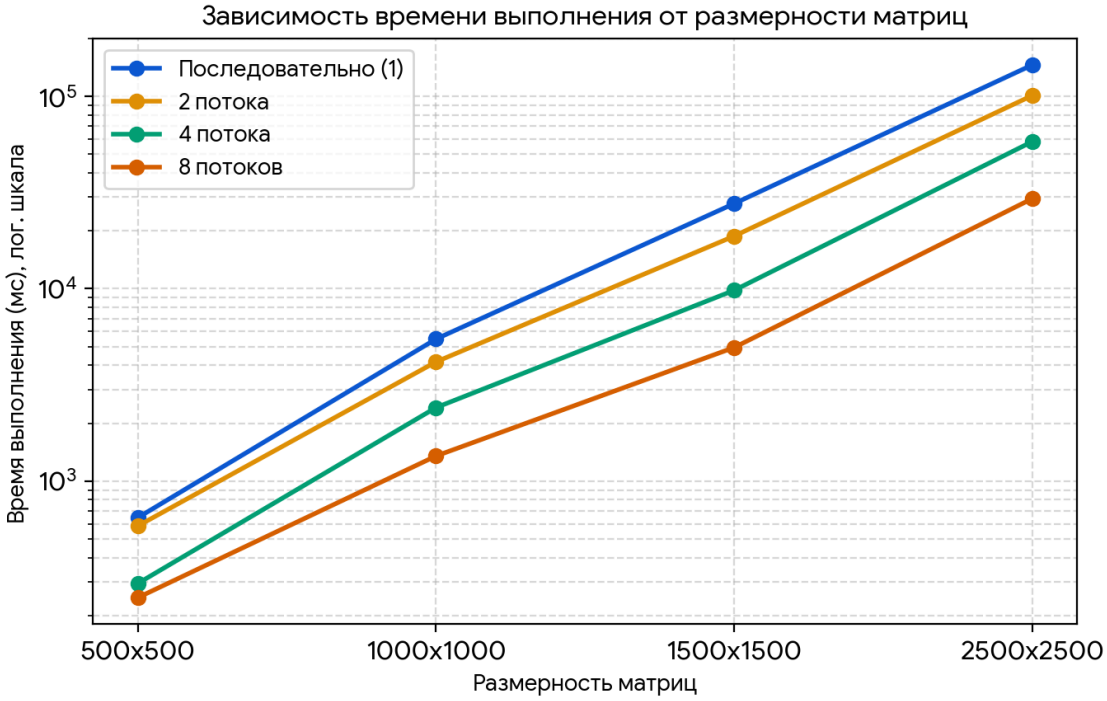


График зависимости времени выполнения от количества использованных потоков при умножении матрицы 500x500:

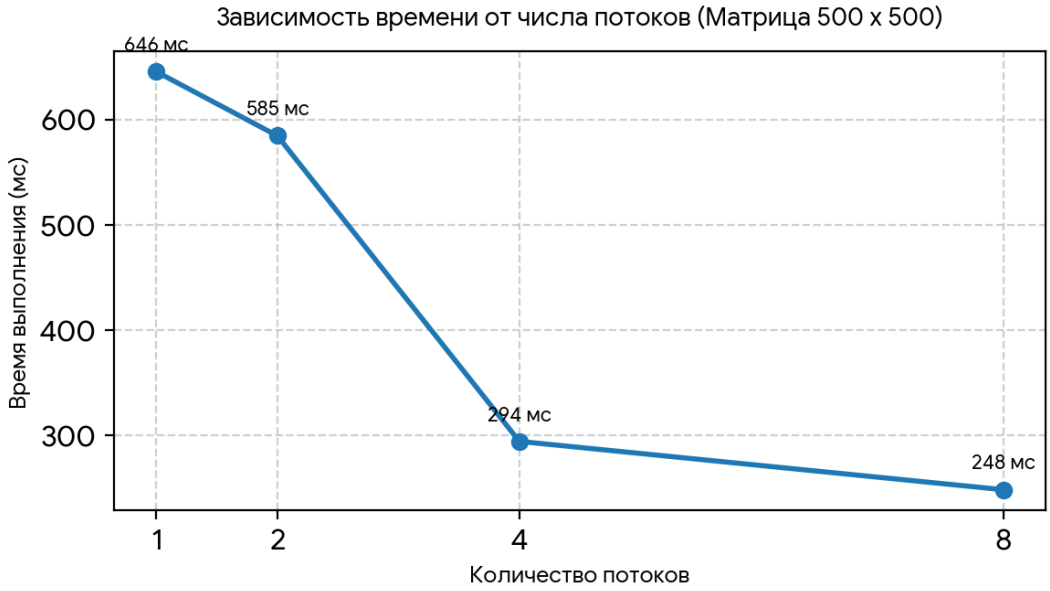


График зависимости времени выполнения от количества использованных потоков при умножении матрицы 1000x1000:

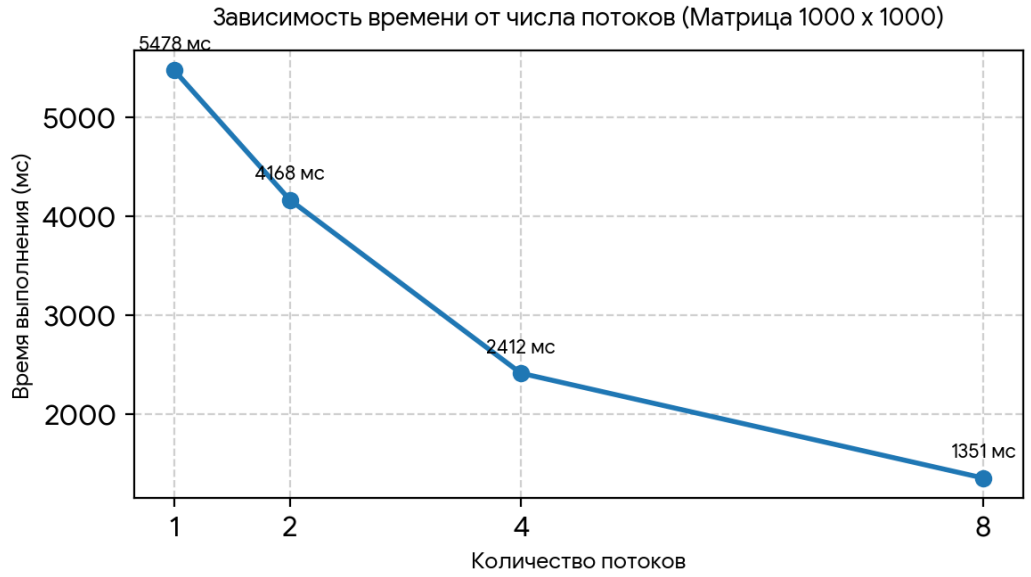


График зависимости времени выполнения от количества использованных потоков при умножении матрицы 1500x1500:

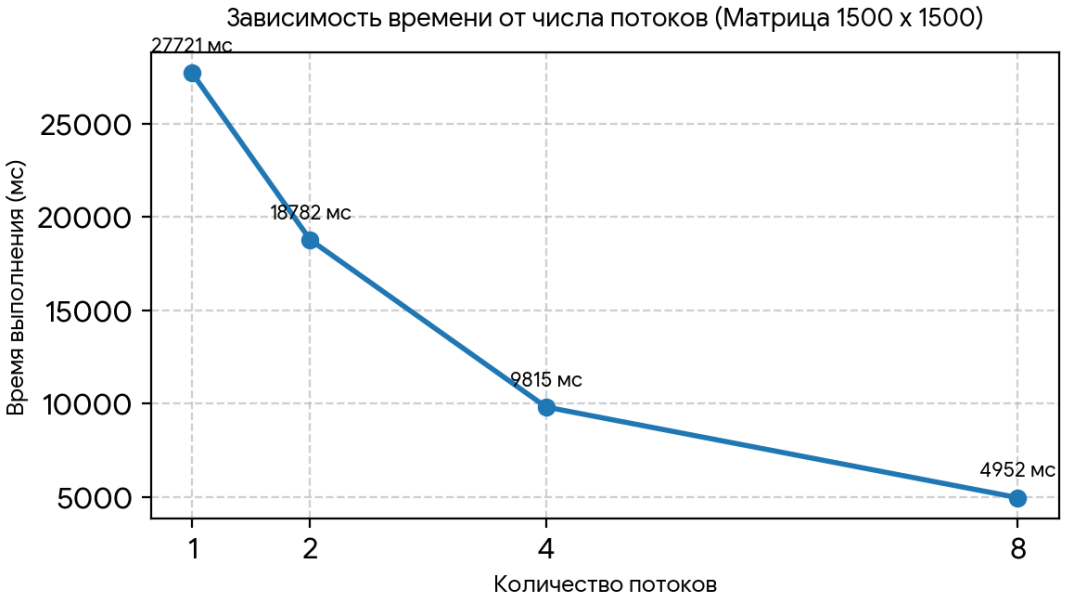
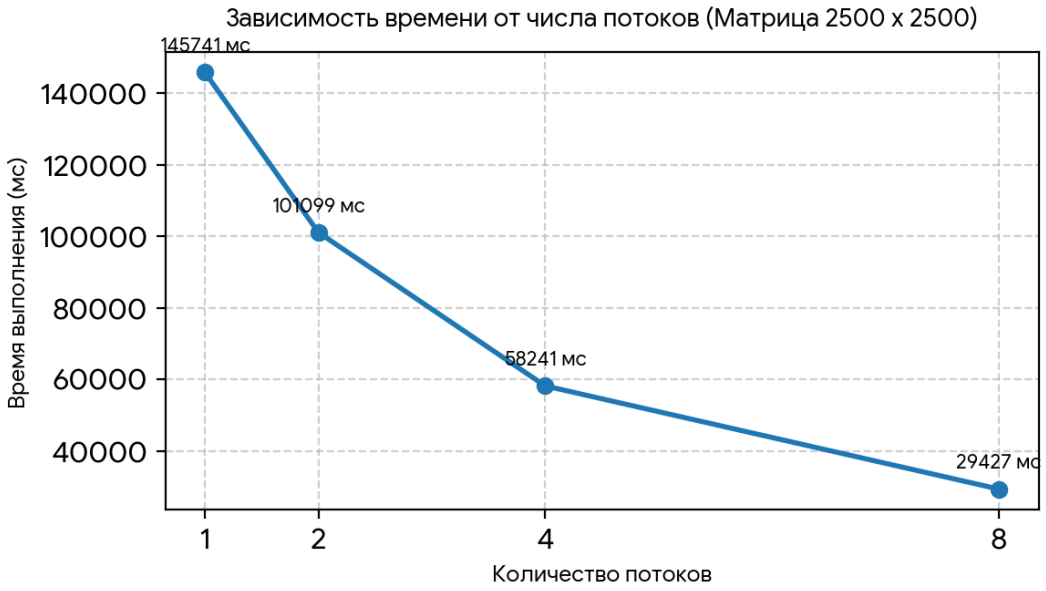


График зависимости времени выполнения от количества использованных потоков при умножении матрицы 2500x2500:



Вывод. В ходе выполнения лабораторной работы был реализован и исследован алгоритм параллельного перемножения матриц на языке C# с использованием библиотеки TPL (Task Parallel Library). Анализ графиков показывает, что эффективность распараллеливания напрямую зависит от объема вычислений:

1) На относительно малых размерностях (500x500) время сокращается незначительно (всего в 2.6 раза при 8 потоках), так как накладные расходы операционной системы на управление потоками сопоставимы с полезным временем счета.

2) На больших объемах данных параллельный алгоритм показывает высокую эффективность, значительно сокращая общее время выполнения задачи (со 145 до 29 секунд). Это подтверждает целесообразность использования многопоточности для тяжелых вычислительных задач.